Sales Visit Planning and Management Platform - Outline

Name: Luke Doyle

Student ID: D00255656

Domain Summary

During my internship, I learned that Field-based sales representatives currently manage their working day across a variety of different diconnected tools. There is no single centralised system for planning visits, scheduling calls, recording outcomes, or tracking expenses. The result is inefficient routing, poor visibility for management, no reliable record of daily activity, and additional labour (as in less working with customers and ore with business administration tasks).

The proposed system aims to address this by giving sales representatives a single platform to plan and manage their working day.
Managers can oversee teams of representatives' and individual representative's activity, flag priority customers, and approve expense submissions.
Administrators handle user (Manager and Sales Rep) account management across the platform.

Three actors operate at the system boundary: the Sales Representative, the Manager, and the Administrator.
An external mapping/location service operates outside the boundary, called during planning to resolve customer addresses for route optimisation.

The long-running processes are day planning, visit and call execution, and expense submission and approval. Where each state involves changes over time and coordination between multiple actors.

Key Actors:

- Sales Representative
- Manager
- Administrator
- External Mapping / Location Service

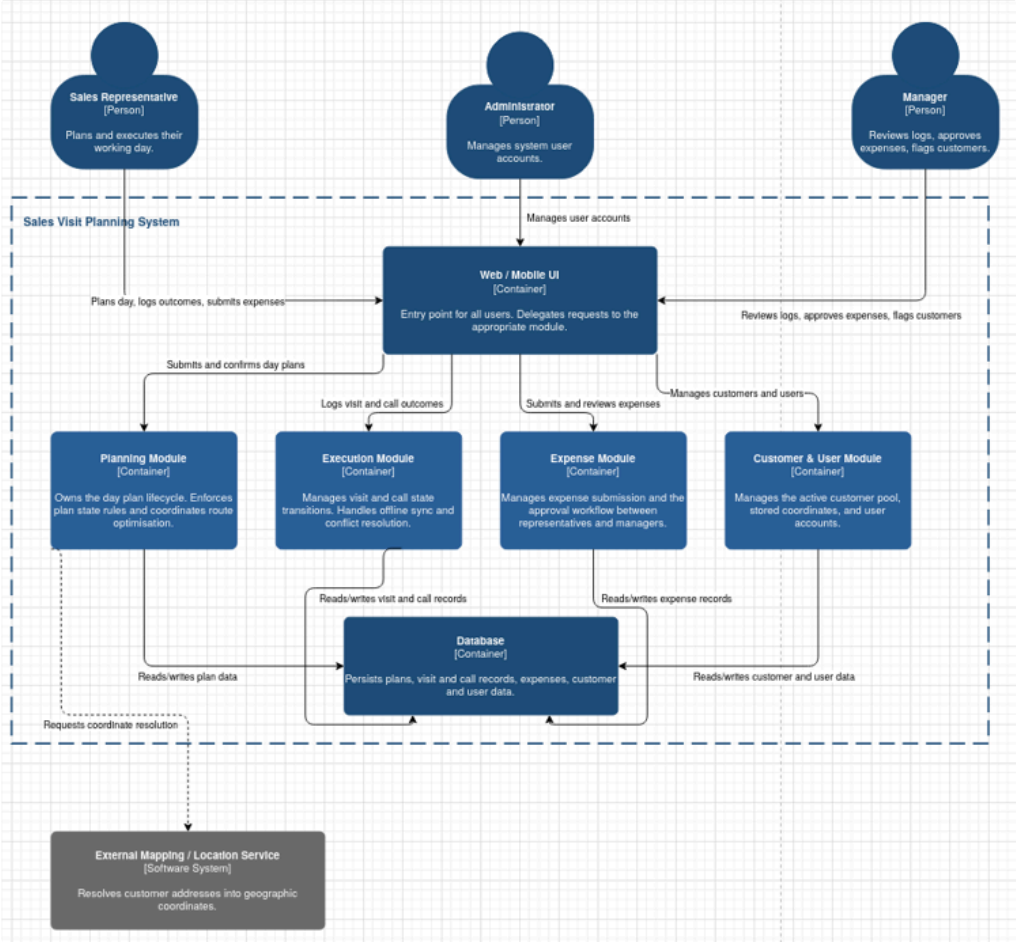
Core Workflows:

- Day Planning
- Visit and Call Execution
- Expense Submission and Approval

Important Business Rules:

- A day plan cannot be confirmed without at least one visit or call scheduled.
- A visit cannot be marked as complete without outcome notes being recorded.
- Expense submissions may only be approved or rejected by a Manager.
- If a customer is deactivated while present on a confirmed or active plan, the representative must be notified, and the affected stop flagged.

High-Level System Structure



Container Summary

Within Boundary:

- Web / Mobile UI (Frontend)
- Planning Module/Service
- Execution Module/Service
- Expense Module/Service
- Customer & User Module/Service
- Database.

Outside Boundary:

- External Mapping/Location Service.

The above five services within the boundary create the system. They all share one database, a single sorce of truth.
One database keeps things consistent and secure, the trade off being the services are not fully data independent, so this sits closer to a modular monolith than true microservices.

The Frontend is the entry point for all three actors. It does not own any domain logic itself. Its job is to take user interactions and send them to the appropriate service. Keeping the UI separate from the other services means domain rules are not scattered across multiple sources. This allows for better reliability and security.

The Planning service owns the day plan lifecycle. It handles plan state transitions, runs the routeing optimisation process, and making API calls to the External Mapping Service when coordinates are not present.

The Execution service owns visit and call state transitions. It handles visit and call outcome logging. It also handles local copies and resolving conflicts when connection is restored. This is separate from the planning service to improve scalability.

The Expense service manages the submission and approval workflow.
Keeping it seperate from execution keeps the financial approval process isolated given the different actors and purpose of the service. A manager approving expenses does not need to interact with the same service a representative uses to log a visit outcome or plan their day. This follows the separation of concerns principle and provides better security later on.

The Customer & User service manages the active customer pool, stored coordinates, and user accounts. Grouping these together means that both customers and users are “reference data” that the rest of the system reads from, rather than transactional records being actively worked through. This does not mean the data is ‘read-only’.

Domain Structure

The domain breaks into four areas: Planning, Execution, Expense Management, and Customer Management.
Each a distinct boundary protecting a different set of rules and lifecycle states.

Planning

The Day Plan is the aggregate root for this area. It owns the plan lifecycle from Draft through Confirmed to Active. The core idea is that a plan cannot move from Draft to Confirmed without at least one visit or call scheduled.
This rule belongs inside the planning boundary because nothing outside it should be able to produce a confirmed plan that violates it. The Day Plan also owns the ordered list of visits and calls that make
up a plan, and the start and end locations used for route optimisation (if selected).

Execution

Each Visit and each Call is treated as its own entity within this area.
A Visit transitions through Planned, In Progress, and then Completed, Missed, or Rescheduled.
A Call transitions through Scheduled, Attempted, and Completed.
The key point is that a Visit cannot be marked as Completed without outcome notes recorded against it.
This rule sits inside the execution boundary rather than the planning boundary because it only applies at the point of execution, not at the point of planning.

Expense Management

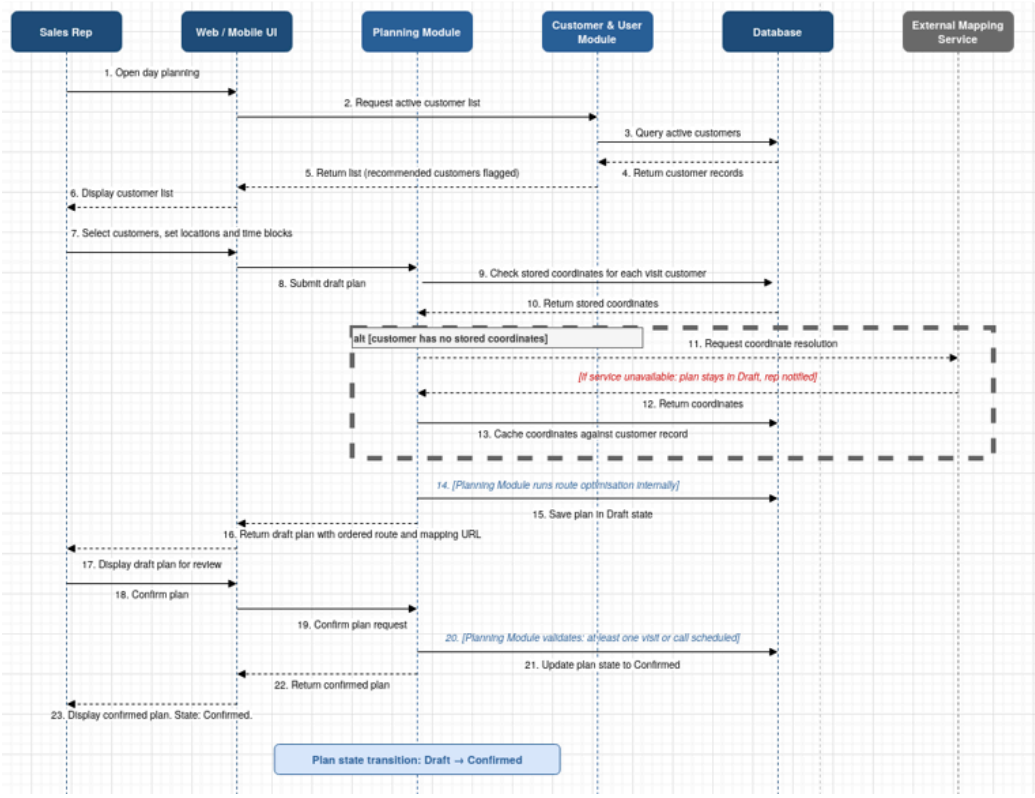
The Expense Submission is rooted here. It transitions through Submitted, Pending Approval, and then Approved or Rejected. A rejected submission can be amended and resubmitted, which returns it to Submitted. The invariant is that only a Manager can move a submission to Approved or Rejected. No other actor can cross that boundary.

Customer Management

The Customer record is rooted in this area. It holds the customer's status, address, and cached coordinates.
The key rule this boundary protects is that coordinates are always stored against the customer record, so any plan involving that customer does not need to call the external service again.
A secondary concern this boundary owns is what happens when a customer is deactivated while present on an active plan.

Workflow and Behaviour Modelling

Day Planning



Workflow Description

The day planning workflow kicks off when the Sales Rep opens the planning screen.

Step 1: Customer List - The Frontend requests the active customer list from the CU service, which queries the database and returns the records, with any manager flagged customers marked up.

Step 2: Plan Setup - The rep picks who to visit and call, sets a start and end location, and assigns time blocks to each call. Calls default to 30 minutes.

Step 3: Coordinate Check - The Planning service checks stored coordinates for each visit customer. If coordinates are there, they get used. If not, it calls the External Mapping Service. Any coordinates returned get cached against the customer record so future plans skip the call.

Step 4: Route Optimisation - Route optimisation runs internally across the visit customers. They get ordered accordingly and a map URL is produced for navigation. The plan saves to the database as Draft and is returned to the rep for review.

Step 5: Confirmation - The rep reviews and confirms. The Planning service enforces the one rule that matters here: at least one visit or call must be on the plan. If that is not met, the plan

stays in Draft. If it is, the plan moves to Confirmed in the database and the UI reflects that.

Failure Path

If the External Mapping / Location Service is unreachable when coordinates are needed, the plan cannot be completed for the affected customer. The plan remains in Draft state. The representative is notified and must either wait for the service to recover, or remove the affected customer from the plan. The workflow does not proceed to Confirmed until the issue is resolved.

If the representative submits a plan with no visits or calls scheduled, the Planning service rejects the confirmation request. The plan remains in Draft, and the representative must add at least one item before confirming.

Consistency and Coordination Thinking

Where consistency is critical:

Consistency Point 1: Plan Confirmation
A plan with nothing scheduled cannot be confirmed. That check lives inside the Planning service at the point of the state transition. Nothing outside it can produce a confirmed plan that skips it.

Consistency Point 2: Expense Approval
Only a Manager can move a submission to Approved or Rejected. That transition is locked inside the Expense service. No other actor, no other path.

Consistency Point 3: Visit Completion
A visit cannot reach Completed without outcome notes recorded against it. Enforced inside the Execution service at the point of transition. Cannot be satisfied from outside that boundary.

Where temporary inconsistency is acceptable:

Acceptable Point 1: Manager Viewing Logs
A manager viewing visit and call logs does not need live state. A rep marking a visit as Completed while the manager has the screen open is fine. It catches up. No consistency risk to the underlying data.

Acceptable Point 2: Coordinate Caching
There is a small window after the External Mapping Service responds where the cache write is in progress. Another part of the system reading the customer record in that window and seeing old state is not a problem. Coordinates are only needed at plan creation time.

Coordination reactions:

Reaction 1: Customer Deactivation
When a customer is deactivated while on a confirmed or active plan, the Customer & User service notifies the Planning service. The affected stop gets flagged and the rep is notified. The plan is not automatically changed. The rep decides what to do.

Reaction 2: Offline Sync
When the Execution service receives locally logged changes on reconnection, it reconciles them against the current server state. Where conflicts exist they must be resolved before anything gets committed.

Behavioural Reactions

Customer deactivation while present on an active plan

When a Manager deactivates (should not remove) a customer through the Customer & User service, the system cannot simply update the customer record and stop there.
If that customer is present on a confirmed or active day plan, the Planning service must be informed. The affected stop is flagged on the plan, and the representative is notified. The plan is not automatically amended or cancelled. The representative retains control over how to respond.

Visit marked as completed

When the Execution service records a visit as Completed, the outcome becomes visible to the representative's Manager through the management view. The Execution service does not directly update a management dashboard or push a notification. It writes the completed record to the database, and the manager's view reflects that when next loaded. This keeps the Execution service focused on state transitions rather than on who is watching them.

Expense submission rejected

When a Manager rejects an expense submission through the Expense service, the representative must be informed so they can amend and resubmit. The rejection state change is owned by the Expense service, but the downstream effect is that the representative's view of that submission must reflect the new state and indicate that a resubmission is possible.

Failure Considerations

Failure Scenario 1: External Location Service Unavailable

If the external mapping service is unreachable, the system cannot resolve coordinates for customers without cached location data, leaving the day plan in an incomplete state.

This is an ongoing degraded state rather than a one-off error. The plan cannot proceed until the service recovers, coordinates are manually provided, or the affected customer is removed.

Failure Scenario 2: Incorrect Coordinates Returned

The external service may return inaccurate coordinates for an ambiguous or poorly formatted address. As these are cached back to the database on return, the error silently affects all future route calculations for that customer until identified and corrected.

Failure Scenario 3: Connectivity Loss Mid-Route

A representative may lose connectivity while working through their plan, leaving them unable to log outcomes or expenses. A local device record of the confirmed plan allows continued operation in a degraded state, with changes synchronised on reconnection. Where the server state has changed during the offline period, conflict resolution is required.

Design Decision Summary

The system architecture is modular in structure with shared UI sitting behind four domain-aligned services writing to a shared database. Not at the level of microservices given the single database, it would be closer to “Mono-Monolithic”.

The primary justification of this architecture is the great diversity of the responsibilities and consistency requirements in the four domains.
Since they are kept as four separate services, they have their own rules and logic with little to no reliance on the core functionality of the neigherbouring serivce, just the inputs and outputs.

At present, the most uncertain part of the design is the “Execution” service and its offline capability. Due to the need for field representatives to work without reliable access to network connectivity, I feel that it is necessary for the device to hold a local copy of the confirmed plan.

Two designs that require further investigation is; “How a Local save is stored on a device?” and Conflict resolution logic on local save vs cloud save post reconnection.

Non-Functional Considerations

Reliability

The most significant reliability concern in the system is the dependency on the External Mapping Service. The design reduces exposure to this by caching coordinates after the first successful resolution. Over time, as more customer records accumulate stored coordinates, the proportion of planning requests that require an external call shrinks. This makes the system more reliable over time as the number of records stored are more then the potential addition of new customers. This would only be false in the case of a massive company overhaul, making the application go back to square one.

Maintainability

Each service has a clearly defined responsibility. A change to expense approval does not touch planning or execution logic. A change to how coordinates are managed does not affect visit logging. This makes it clear where a given rule lives and reduces the risk of conflicting enforcement building up across the codebase over time.

Scalability

The current design makes no strong scalability claims. A shared database and modular structure within a single deployment suits the expected scale of a field sales team. If user numbers grew significantly, the service boundaries already in place make it more straightforward to extract individual services without restructuring the domain logic.